Concurrency-Aware Permission Decisions in Multi-Tenant LLM Agent Runtimes

Mingyuan Xie*
Qingdao University of Technology, Qingdao, Shandong, China
jhj208436@gmail.com

Zhengxun Wu
Qingdao University of Technology, Qingdao, Shandong, China
2276815088@qq.com

Abstract

Multi-tenant agent runtimes multiplex tenant work, permission checks, and audit activity over finite executors. We characterize a concurrency failure in which tenant-induced queueing delays a permission decision until its deadline, and a permissive timeout fallback converts executor unavailability into an unsafe admission. We present Neuron, a Java runtime architecture that reserves permission capacity and exports a resource-pressure signal to the escalation policy. Five independent Java 21 JVM runs on a Windows host and an Ubuntu 22.04 one-vCPU KVM VPS cover 71,880 observations across deadline, attacker-pressure, and executor-capacity sweeps. Four predeclared architectures separate timeout semantics, permission-pool isolation, and cross-layer coupling. We additionally replay 33 API-derived sessions (254 immutable tool calls), exercise a process-separated Java permission service over same-host loopback with injected delay, queue pressure, and pauses (3,200 observations), and evaluate a post-selection holdout of 180 observations from six unseen workload profiles. An additional Windows core run contributes 3,240 executor-focused observations at the same factor grid. We further perform a 2x2 capacity/signal ablation over 1,800 observations from 5 independent JVM runs. Shared fail-open execution loses decision availability under burst-lock pressure; reserved capacity restores timely decisions, while coupling changes the depth-1 verdict when pressure exceeds the fixed threshold of 50. The holdout detects 90/90 attack observations with 0/90 false positives. The loopback experiment is not evidence about WAN or multi-node deployments.

Keywords: Concurrent runtimes Multi-tenant systems Executor contention Permission decisions Cloud services LLM agents

1 Introduction

Multi-tenant LLM agents share runtimes, tool gateways, caches, and execution capacity across tenants [1, 2]. This consolidation makes permission checking a concurrency problem. A permission decision is scheduled on an executor, reads policy state, may append an audit event, and must complete while untrusted tenants generate work. Queueing, lock occupancy, and executor rejection can therefore change the behavior of a permission path even when resource admission and authorization are implemented as separate modules. This interaction between concurrency and runtime control is the focus of this paper.

*Corresponding authors: Mingyuan Xie and Zhengxun Wu.

We study one conditional but consequential failure mode. Suppose a runtime sets a deadline for a permission decision and uses a permissive result when that deadline expires. Hostile work that delays a shared policy executor can then turn an executor-availability failure into an unsafe admission. The condition is not inevitable: fail-closed handling preserves safety, and a reserved permission executor can avoid the scheduling bottleneck. The point is that deadline semantics, capacity allocation, queue pressure, and resource signals must be evaluated together rather than hidden behind separate layer interfaces.

Neuron. We present **Neuron**, a concurrency-aware runtime architecture with two core mechanisms. First, permission checks execute on reserved capacity instead of the executor used by tenant work. Second, resource admission exports a pressure signal to the escalation policy; under sustained pressure, a destructive request one level below the normal depth boundary is denied instead of entering an interactive approval path. A per-turn trace identifier joins the resource event and permission verdict for audit correlation. Spawn-time identity propagation and privilege non-increase support the design, while quotas and sandboxes are implementation mechanisms rather than co-equal research contributions.

Systematic evaluation. An earlier draft used Python multiprocessing to model Java lock contention and emphasized one 3 ms deadline. That model demonstrated a possible mechanism but could not establish behavior of the Neuron runtime. We remove it from the evidence chain. The present study executes the Java 21 runtime `RateLimitSyscallFilter` and `EscalationPolicy` classes in Java 21, retains immutable JSONL observations, and treats each independently started JVM as the replication unit. Five core runs are completed on each of two environments, and a separate capacity experiment varies the permission pool and queue. The four predeclared architectures are:

1. a shared permission executor with fail-open timeout handling;
2. the same shared executor with fail-closed handling;
3. a reserved permission executor without cross-layer coupling; and
4. reserved capacity plus Neuron's pressure-aware joint decision.

The comparison separates two effects. If both reserved configurations meet their deadlines, reservation explains availability. If only the coupled configuration changes a depth-1 destructive request from ASK to DENY, the difference is attributable to the resource signal rather than executor isolation. A fixed-trace replay of previously collected API sessions then tests the same decision path with application-level tool-call sequences; it is a replay, not a new claim of live deployment.

Research questions. The paper follows the causal chain "failure exists → mechanism is isolated → behavior is checked outside one machine."

RQ1 Under what combinations of permission deadline, executor load, and lock-hold time does a shared Java policy executor miss its deadline, and how do timeout and capacity configurations differ?

RQ2 Does a resource-pressure signal change the escalation policy verdict beyond the effect of reserving executor capacity?

RQ3 Do the queueing and availability patterns reproduce on a second containerized environment, and how sensitive are they to permission-pool and queue capacity?

RQ4 Does the availability trade-off persist when the permission policy runs in a separate Java process behind an HTTP service with injected delay and pause faults?

RQ5 Does the concurrent decision path preserve completion for benign and adversarial API-derived tool-call traces, and what residual risks remain outside the local JVM/container boundary?

Contributions. This work contributes:

1. a precise concurrency condition for executor-induced permission unavailability that states the required deadline and timeout semantics;
2. a runtime architecture coupling permission-executor capacity with a resource-informed permission verdict; and
3. a reproducible Java 21 evaluation with four fair baselines, two environments, independent JVM replications, deadline/load/capacity sweeps, automated raw-data aggregation, and a fixed API-trace replay.

Identity propagation, privilege non-increase, and audit correlation are supporting mechanisms. WASM, Docker, quota enforcement, and LLM routing are engineering context and are reported in the artifact or appendix rather than presented as independent innovations. AutoGen, LangGraph, and MCP are treated as orchestration/protocol examples, not as competing multi-tenant security systems; observations about their defaults therefore motivate integration requirements but do not establish superiority over those projects.

Section 2 defines the failure conditions. Section 3 states assumptions and non-goals. Section 4 describes the architecture. Section 5 reports the cross-environment evaluation, Section 6 positions the work, and Section 7 concludes.

2 Background and Motivation

This section establishes the conceptual baseline of multi-tenant LLM agent runtimes (Section 2.1), exposes the limitations of conventional, layer-separated isolation (Section 2.2), characterizes the novel attack surfaces introduced by LLM agents (Section 2.3), and derives the design goals that motivate Neuron (Section 2.4).

2.1 Multi-Tenant LLM Agent Runtimes

A multi-tenant LLM agent runtime extends the classical process abstraction to agent workloads. Where a conventional process binds a program counter and a file-descriptor table to a scheduling entity, an agent process binds a *turn*---an atomic interaction comprising a prompt, a sequence of tool calls, and an LLM-mediated control flow---to a runtime-managed execution context. The runtime is responsible for scheduling turns, mediating tool execution, accounting for resource consumption (principally LLM tokens, which play the role of CPU cycles), and enforcing isolation among co-resident agents.

Three properties of modern agent workloads distinguish them from conventional processes and elevate them to first-class runtime concerns:

- **Long-running and stateful.** Agents persist memory, plans, and intermediate artifacts across turns and sessions, creating durable state that must be isolated and accounted for over long horizons.
- **Dynamic and self-spawning.** An agent may, in the course of a single turn, spawn sub-agents to delegate sub-tasks. The resulting spawn tree is constructed at runtime from natural-language decisions, not from a static process graph.
- **Multi-tenant.** A single runtime instance concurrently serves multiple tenants that share low-level substrates---an event bus for inter-agent communication, a virtual file system (VFS) for artifact storage, and a pool of virtual threads for execution---while requiring strict isolation and per-tenant fairness.

Existing frameworks address these properties at the application layer. AutoGen [2] and LangGraph [3] provide orchestration primitives but delegate resource accounting to user-space token counters and access control to static policy files. The Model Context Protocol [4] standardizes tool exposure yet is silent on resource governance. In all cases, the governance machinery lives above the runtime boundary, observing the workload from the outside.

2.2 Limitations of Layer-Separated Isolation

Classical isolation designs separate concerns across three largely independent layers, each with its own mechanism and its own audit trail:

- **Resource control** (quota managers such as Linux cgroup v2 [5]) accounts for and caps resource consumption---CPU, memory, and, in our setting, LLM tokens. Resource decisions are recorded in quota-manager-local logs.
- **Execution isolation** (sandboxes, seccomp, containers such as gVisor [6] and Firecracker [7]) confines what an execution context can do at the system-call level. Isolation decisions are recorded in sandbox-local logs.
- **Access control** (capabilities [8, 9], declarative policies) determines whether an operation is permitted. Permission decisions are recorded in permission-local logs.

This separation is benign when the three layers are genuinely independent. It becomes *unsafe* under multi-tenant agent workloads because the layers share a common resource pool and a common execution substrate. Two coupled pathways, invisible to any single layer, undermine the entire design:

Security starvation under resource contention. The permission layer is not a free abstraction; it consumes CPU to evaluate rules, memory to retain audit state, and locks to coordinate concurrent decisions. When a malicious tenant mounts a resource-contention attack---for example, flooding the event bus with broadcasts or hammering the VFS with writes---the security module is itself a victim of the contention it is supposed to police. Under sufficient pressure, policy evaluation is delayed, audit buffers overflow, and permission checks degrade to permissive fallbacks. The security layer thus fails *because of* a resource condition that it has no mechanism to observe, let alone act upon.

Privilege escalation in contention windows. The converse pathway is equally dangerous. Between the moment resource pressure begins to build and the moment the resource layer trips a hard quota-exceeded signal, there exists a transient *contention window* in which the runtime is impaired but not yet in failure. During this window, an attacker can issue a privileged operation---a cross-tenant VFS access, a destructive tool call---that the permission layer, still nominally functional, admits because it has no visibility into the resource state and no causal link to the ongoing attack. By the time either layer logs the event, the two records sit in disjoint trails with no shared identifier to correlate them.

The root cause is the absence of a *shared causal identifier*. Without a single trace that threads through quota-manager, sandbox, and permission decisions, cross-layer correlation is impossible: the runtime cannot reconstruct that "the permission grant at time t occurred inside the contention window opened by the broadcast flood at time $t - \delta$." Each layer operates---and fails---in isolation.

2.3 Novel Attack Surfaces in LLM Agents

Beyond the coupled-failure mode, LLM agents introduce attack surfaces that classical isolation models were not designed to handle, because they violate two foundational assumptions: that privileges bind at start-up, and that escalation proceeds through explicit system calls.

Spawn-time privilege escalation. In a classical system, a child process inherits a subset of its parent's privileges; a downgraded parent cannot produce an upgraded child. In an agent runtime, a sub-agent is constructed by instantiating a fresh execution context through a tool call. If the spawn path does not explicitly propagate the parent's effective privilege profile, the child defaults to a permissive baseline---*spawn itself becomes an escalation*. A parent agent that has been deliberately downgraded (for example, a reviewer restricted to read-only tools via a `*:deny` profile) can thereby produce a child with full default privileges, defeating the downgrade in a single tool call.

Tenant-identity loss across spawn boundaries. Multi-tenant isolation depends on every execution context carrying a tenant identifier that the access-control layer can check. If the spawn path neglects to propagate the tenant identity to the child, the child's tenant field is null, and tenant-ownership checks---which conservatively skip when the caller's tenant is unknown---silently permit cross-tenant access. A sub-agent spawned by a tenant-A parent can then read or mutate tenant-B artifacts.

Natural-language privilege escalation without spawn-tree context. An agent requests permission to invoke a sensitive tool through a structured approval channel. If the request payload carries only the agent identifier, the tool name, and the target---but not the position of the requesting agent within the spawn tree---then a human approver (or an automated policy) cannot distinguish a top-level request from one originating at depth three. A deeply nested sub-agent, precisely the context in which an attacker is most likely to attempt social-engineering escalation, is indistinguishable from a benign top-level call. The approval channel, designed for flat process models, becomes the weak link.

Resource-contention flooding. Finally, the shared substrates themselves offer an unthrottled attack surface. Without per-tenant rate limits at the resource layer, a malicious agent can sustain high-frequency event-bus broadcasts and VFS writes, consuming SSE fan-out capacity and lock contention budget *before* the permission layer has any opportunity to intervene. The attack succeeds entirely within the layer-separated blind spot.

2.4 Governance Gap and Design Goals

The foregoing analysis reveals a single governance gap with two facets: the layers are *causally disconnected* (no shared identifier), and the spawn boundary is *semantically unguarded* (privileges and tenant identity are not enforced to be non-increasing). From this gap we derive the design goals that Neuron must satisfy:

G1. Unified causal audit. Every governance decision---resource, isolation, or permission---must carry a single per-turn trace identifier and be aggregated into a unified audit chain, so that cross-layer correlation and real-time attack truncation become possible.

G2. Spawn-time privilege non-increase. A child agent's privileges must be a subset of its parent's; spawn must never constitute an escalation, closing the attack surface of Section 2.3.

G3. Tenant isolation across spawn. Tenant identity must propagate across spawn boundaries, and tenant-ownership checks must execute as a hard pre-check that cannot be bypassed by permissive modes.

G4. Depth-aware escalation. Every approval request must carry the spawn-tree context (depth and parent chain), and destructive operations originating from deeply nested sub-agents must be denied automatically.

G5. Resource-layer throttling. The shared substrates must enforce per-tenant rate limits that act *before* the permission layer, so that contention is damped at the source rather than policed after damage.

G6. Zero regression. All closures must be transparent to headless execution, preserving existing fallback contracts so that the runtime's correctness test suite remains green.

Goals G1 and G5 target the coupled-failure mode of Section 2.2; goals G2--G4 target the LLM-specific attack surfaces of Section 2.3; G6 constrains the design space to evolvable, non-disruptive closures. The Neuron architecture presented in Section 4 is designed to satisfy these goals jointly, rather than in the layer-separated fashion that motivates this work.

3 Threat Model

Neuron protects a multi-tenant agent runtime against tenants that use valid runtime interfaces adversarially. The model distinguishes the attacker, deployment choices required for the studied failure, and the trusted computing base. This distinction is necessary because resource pressure is not by itself an authorization bypass.

3.1 Attacker and Assets

The attacker controls one or more tenant accounts and all prompts, tool requests, and sub-agent creation requests issued by those accounts. The attacker may coordinate concurrent clients, measure its own response times, vary request rate, and choose workloads that occupy shared executors or locks. It knows published limits and may use bursts or low-rate, long-hold tasks. It cannot directly mutate another tenant's state, the policy code, or the runtime's counters.

The protected assets are (i) the integrity of permission verdicts, (ii) availability of the policy path for benign tenants, (iii) tenant and privilege identity across dynamic spawn, and (iv) an audit record that links resource admission to the permission decision. Confidentiality of model weights and prompts is important but is not the outcome measured in this paper.

3.2 Failure Preconditions

The fail-open security failure requires all of the following conditions:

1. tenant work and permission work contend for a finite shared resource, such as a bounded executor;
2. the deployment imposes a finite permission-decision deadline;
3. a request that misses that deadline receives a permissive result; and
4. the attacker can generate enough admitted work to delay the policy task beyond the deadline.

Removing any one condition changes the outcome. A fail-closed fallback preserves the security verdict while potentially denying legitimate work. A reserved or remote policy service can remove the relevant shared bottleneck. Admission control can prevent hostile work from reaching it. The evaluation therefore treats fallback semantics and capacity isolation as factors, not as hidden properties of a baseline.

Neuron's joint rule addresses a related but distinct situation. A request may complete on time yet arrive during sustained resource rejection. The attacker can request a destructive tool at depth 1, where the static policy would normally ask for approval. If the resource signal crosses the frozen threshold, the coupled rule returns DENY-DEPTH. This guarantee assumes the pressure counter is authentic and scoped to the same tenant or causal episode.

3.3 Trusted Computing Base

The JVM, Neuron policy and admission code, host kernel, and container runtime are trusted. The attacker does not have host administrator access, cannot attach a debugger to the JVM, and cannot rewrite the running bytecode or raw audit files. These assumptions are conventional for a runtime reference monitor. Compromise below this boundary voids the claim.

Tools capable of arbitrary native execution must be confined by an external sandbox or container. The broader Neuron prototype contains sandbox providers, but this paper does not evaluate container escape resistance. Consequently, the security result is about permission admission before tool execution, not containment after a malicious native tool has started.

3.4 In-Scope Actions

Within the boundary, the attacker may:

- flood valid runtime operations at configurable concurrency and rate;
- choose work whose service time increases executor occupancy;
- attempt destructive tool requests at different spawn depths;
- spawn sub-agents and try to obtain a more permissive profile; and
- distribute requests across several client threads belonging to the same tenant identity.

The runtime must preserve tenant identity and non-increasing privilege at spawn. For the central experiment, it must also apply the declared timeout semantics and record whether the policy task was rejected at queue admission or expired after admission.

3.5 Non-Goals

Neuron does not prevent prompt injection, model jailbreaks, poisoned model weights, or malicious content generation. It assumes that a dangerous request can reach the policy gate and evaluates enforcement from that point. It does not address cache, power, or microarchitectural side channels. It does not stop colluding tenants from communicating out of band, and the current pressure counter is not proven robust to a Sybil attack that creates many separately accounted tenants.

The paper also does not claim Byzantine audit storage, distributed consensus for policy state, or availability during host compromise. Multi-node policy services, GPU schedulers, and remote partitions require a different failure model. Finally, the controlled workload is not a substitute for production traffic: it identifies causal boundaries and response surfaces, while a real deployment study is needed for prevalence and capacity planning.

3.6 Security Properties

Within the stated boundary, the design targets four properties:

Timeout transparency. Every missing verdict is distinguishable as queue-admission failure or post-admission deadline expiration, and the configured fallback is applied explicitly.

Policy-plane reservation. Tenant work cannot consume the workers reserved for permission decisions.

Pressure-conditioned tightening. For a destructive depth-1 request, an authentic pressure value above the selected threshold changes the verdict from ASK-WITH-CONTEXT to DENY-DEPTH.

Privilege non-increase. A spawned child receives no permission that is absent from its parent's effective profile.

The first three properties are exercised in Section 5. The fourth is an architectural invariant supported by regression tests but is not part of the starvation response-surface result.

4 Neuron Architecture

Neuron is middleware between an agent framework and host/container services. The architecture makes one research contribution---a coupled resource and permission decision path---and separates it from supporting safety invariants and prototype engineering. Figure-level breadth is deliberately avoided: quota trees, sandboxes, model routing, and audit storage are not presented as independent innovations.

4.1 Control-Plane Boundary

Tenant agents issue typed runtime requests containing a tenant identifier, agent identifier, turn/trace identifier, spawn depth, action, and payload. The dispatcher first applies resource admission, then evaluates permission, then dispatches an admitted operation to a tool or sandbox. The order is security-relevant: rejected tenant work should not occupy permission workers, and an action cannot execute before a permission verdict exists.

Neuron distinguishes two execution domains:

Tenant domain. Agent turns, model callbacks, and tool preparation may be concurrent and are assumed adversarial. Their executor and queues are capacity limited.

Policy domain. Permission evaluation uses reserved workers and a bounded queue. It consumes immutable request context plus authenticated resource state. The core runtime prototype uses an independent JVM executor; the fault-injection artifact also realizes this boundary as a separately started Java process behind a loopback HTTP service.

Executor reservation is not coupling. It is a prerequisite for a dependable policy plane. Coupling occurs only when resource state becomes an input to the permission function.

4.2 Core Mechanism: Resource-Informed Permission

Let d be spawn depth, a the requested action, D the normal destructive action boundary, r the resource-layer rejection count in the configured window, and τ the pressure threshold. Let $\mathit{destructive}(a)$ be the runtime's action classification. The static policy is

$$P_0(d, a) = \begin{cases} \textsc{deny-depth}, & d \geq D \wedge \mathit{destructive}(a), \\ \textsc{ask-with-context}, & \text{otherwise.} \end{cases} \tag{1}$$

Neuron adds a pressure-conditioned clause:

$$P_c(d, a, r) = \begin{cases} \textsc{deny-depth}, & \mathit{destructive}(a) \wedge d \geq D \\ & \vee\ \mathit{destructive}(a) \wedge r > \tau \wedge d \geq D - 1 \\ \textsc{ask-with-context}, & \text{otherwise.} \end{cases}, \tag{2}$$

The rule tightens by one level; it never relaxes a static denial. In the prototype $D = 2$. Thus a depth-2 destructive request is denied with or without coupling. A depth-1 request is the discriminating probe: it is ASK-WITH-CONTEXT when $r \leq \tau$ and DENY-DEPTH when $r > \tau$. Non-destructive actions are unaffected by this rule.

The Java runtime's `EscalationPolicy` implements this function. The resource signal is produced by the runtime's `RateLimitSyscallFilter`, a tenant-scoped token bucket. The current signal is deliberately simple and has limitations: a cumulative count requires an explicit reset/window policy, may not transfer across hardware, and may be diluted by Sybil tenants. The evaluation therefore freezes the counter semantics and treats τ as a deployment parameter.

4.3 Policy Availability and Timeout Semantics

A caller submits the immutable permission task to the policy executor and waits for a configured deadline. There are three possible outcomes:

1. the queue accepts the task and the policy returns before the deadline;
2. the queue rejects the task because it is full; or
3. the queue accepts the task but it does not finish before the deadline.

Outcomes 2 and 3 are distinct telemetry events but require the same explicit deployment choice. A fail-open wrapper returns a permissive result; a fail-closed wrapper denies. Neuron's recommended configuration is reserved capacity plus fail-closed timeout handling. The fail-open configuration is retained only as an experimental baseline because it exposes the security consequence under study.

Reservation has two parameters: worker count and queue capacity. Too little capacity denies legitimate bursts; an unbounded queue converts overload into unbounded latency and memory

consumption. The prototype consequently uses a bounded queue and records its depth with every observation. The process- separated fault-injection path provides a stronger CPU/memory boundary but introduces RPC and partition semantics; the current experiment keeps the service on loopback, so WAN-scale behavior remains outside the claim.

4.4 Causal Context and Audit Correlation

Admission and permission events carry the same trace identifier. A resource record includes tenant, action, bucket outcome, rejection count, and time. A permission record includes depth, action class, pressure value consumed, verdict, executor admission result, deadline outcome, and policy latency. The pair answers a question that separate logs cannot answer reliably: which resource state actually informed a particular verdict?

Correlation is supporting infrastructure, not enforcement. If the resource counter is unauthenticated or a trace identifier can be replaced by tenant code, the joint rule becomes attacker-controlled. Neuron creates both inside the runtime boundary and propagates immutable copies to child tasks. Audit failure does not silently convert a denial to an allow; a deployed system must choose whether audit backpressure itself is fail-closed or buffered in reserved storage.

4.5 Supporting Invariant: Privilege Non-Increase

Dynamic spawn introduces a second way to cross the policy boundary. Let $\Pi(x)$ denote the effective capabilities of agent x. For every spawn edge $p \to c$, Neuron requires

$$\Pi(c) \subseteq \Pi(p). \tag{3}$$

The spawn path captures the parent's effective profile before asynchronous dispatch, derives the child profile by intersection with the requested profile, and propagates tenant, trace, and depth in the same immutable spawn context. Missing parent context is not interpreted as a default permissive profile. Cleanup occurs in a `finally` block so pooled threads cannot leak identity to later tasks.

The invariant supports coupled governance because the depth and tenant used by P_c must describe the actual caller. It is nevertheless orthogonal to the starvation claim: a system can preserve privilege inheritance while its policy executor is unavailable. Neuron tests the invariant separately and does not fold those tests into the response-surface sample count.

4.6 Engineering Components and Trust Boundary

The broader prototype includes hierarchical quotas and sandbox providers. Quotas supply admission decisions; sandboxes contain tools after permission has been granted. Neither changes the equations above. For clarity:

- token quotas and token buckets are resource-admission mechanisms;
- a WASM or container provider is an execution-isolation mechanism;
- identity propagation, privilege intersection, and correlated audit are supporting security mechanisms; and
- reserved policy capacity plus P_c is the evaluated coupled governance mechanism.

The container/JVM boundary remains trusted. This paper neither evaluates container escape nor claims that an in-process sandbox withstands arbitrary native code. Newly registered native tools must execute outside the policy JVM under an independently configured sandbox.

4.7 Request Flow

For a destructive tool request, the complete flow is:

1. validate the runtime-authenticated tenant, trace, and spawn context;
2. apply tenant-scoped resource admission and update the pressure state;
3. submit a permission task to reserved capacity;
4. compute P_c from depth, action class, and the captured pressure value;
5. apply fail-closed handling if the task cannot be admitted or misses its deadline;
6. append correlated resource and permission records; and
7. dispatch only an admitted operation to the configured sandbox/tool.

The evaluation manipulates steps 2--5 while keeping the runtime admission and policy functions fixed. This is what permits attribution among load, isolation, fallback semantics, and coupling.

5 Evaluation

The evaluation is designed around the paper's causal claim rather than an inventory of Neuron features. We first reproduce policy-plane starvation in the Java runtime, then separate timeout semantics from executor isolation, and finally test capacity, benign quality of service (QoS), and an application-level trace replay. Quota, sandbox, privilege-propagation, and audit-correlation tests remain regression evidence in the artifact but are not counted as independent contributions.

5.1 Questions and Metrics

RQ1: failure conditions. We measure the fraction of permission requests that do not return a verdict within the configured deadline. For the depth-2 destructive probe, a fail-open timeout is unsafe because it becomes ASK-WITH-CONTEXT; fail-closed handling returns DENY-TIMEOUT. We report timeout rate, secure-decision rate, and p50/p95 latency.

RQ2: isolation versus coupling. Both reserved-capacity configurations use the same permission pool. If both meet their deadlines, reservation explains availability. A difference in the depth-1 destructive probe is attributed to coupling only when the policy code, capacity, and request are otherwise identical. We therefore add a factorial ablation that crosses capacity isolation (shared versus reserved) with the pressure signal (disabled versus enabled), holding fail-closed semantics constant in all four cells.

RQ3: cross-environment and capacity sensitivity. The unit of replication is an independently started JVM, not a request within one process. We report per-run summaries and ranges. A nine-cell local sweep varies permission-pool size in $\{1, 2, 4\}$ and queue capacity in $\{16, 128, 512\}$; the representative deadlines are 10 and 50 ms.

Table 1: Architectural baselines. ‘‘Shared’’ means that tenant and permission work use one bounded executor.

Configuration	Policy capacity	Timeout	Pressure signal
Layer-separated + fail-open	Shared	Permissive	No
Layer-separated + fail-closed	Shared	Deny	No
Isolated, no coupling	Reserved	Deny	No
Neuron coupled	Reserved	Deny	Yes

RQ4: process-separated policy-service behavior. We measure the same safety and QoS metrics when permission evaluation runs in an independently started Java process behind HTTP, with injected service delay, queue pressure, and pause.

RQ5: application QoS and residual risk. We measure benign completion, attack security, error tightening, and p95 latency for 100/0, 90/10, 75/25, and 50/50 benign/attack mixes. We also replay immutable tool-call traces produced by earlier real API sessions. This replay tests integration with application-shaped calls without claiming a new live deployment. Audit-queue exhaustion remains outside this evaluation.

5.2 JVM-Native Experimental Design

Runtime path. The benchmark is compiled with the Neuron Java 21 project. Virtual threads generate tenant requests; bounded platform-thread executors represent shared and reserved capacity. Attacker tasks occupy a worker for a configured hold time. Permission requests invoke the Java runtime's `EscalationPolicy.evaluate`. Coupled runs additionally invoke the runtime `RateLimitSyscallFilter.preFilter` token bucket and pass its rejection count into the policy. Python is used only after execution to aggregate immutable JSONL observations.

Architectures. The four predeclared comparisons use identical request generation, policy code, and measurement logic.

The core configuration uses a shared pool of four workers, a reserved permission pool of two workers, and a queue of 512. Each cell contains 30 decisions after a 100 ms warm-up. The core response surface has five independent runs per host; the capacity sweep is a separate local experiment with the same four architectures. The analysis scripts count 71,880 core/capacity observations from 19 JVM runs on 2 hosts, including 10 core runs and 9 capacity configurations.

Factor sweeps. The cross-environment core run uses 3, 10, and 50 ms deadlines, a fixed burst-lock workload, and attacker counts $\{1, 4, 16\}$. The response surface fixes 10 ms and crosses attacker count, request rate $\{100, 500, 2000\}$ requests/s, and task hold time $\{0.5, 2, 5\}$ ms. The local capacity sweep uses permission pools $\{1, 2, 4\}$ and queues $\{16, 128, 512\}$ at 10 and 50 ms. Every verdict, admission failure, latency, queue depth, and rejection counter is retained.

Concurrency-focused replication. To make the executor interpretation explicit, we independently started one additional Java 21 core run on the Windows host (`ccpe_core_01`). This run repeats the deadline and pressure factors without changing the architecture code or the four baselines. In addition to verdicts and timeouts, the analysis reports offered request rate, estimated worker demand, permission-queue depth, and p95 decision latency. At the highest pressure point (16

Table 2: Executor observables at the highest offered concurrency in the additional Windows core run (10 ms deadline; 32,000 offered requests/s).

Configuration	Timeout	Queue p95	Decision p95 (ms)
Layer-separated + fail-open	100.0%	512.0	19.47
Layer-separated + fail-closed	100.0%	512.0	19.41
Isolated, no coupling	0.0%	0.0	0.53
Neuron coupled	0.0%	0.0	0.28

Table 3: 2x2 ablation at high executor pressure. All cells use fail-closed semantics and a 10 ms permission deadline.

Configuration	Timeout	Queue p95	Decision p95 (ms)
Shared, no signal	100.0%	512.0	14.41
Shared, signal	15.3%	13.1	15.64
Reserved, no signal	0.0%	0.0	0.60
Reserved, signal	0.0%	0.0	0.40

attackers, 2,000 requests/s per attacker, 5 ms hold time), the shared executor reaches its 512-entry queue and times out, while both reserved-capacity configurations keep the measured permission queue empty.

The new run contains 3,840 observations and is an independent JVM replication, not a second physical architecture. The result isolates the concurrency mechanism: shared capacity accumulates queueing before the permission deadline, whereas reservation removes tenant work from the permission executor. Coupling changes the verdict semantics once the resource signal is high; it is not credited with the queueing improvement.

2x2 capacity/signal ablation. The ablation uses the same Java 21 executor path and crosses shared versus reserved permission capacity with the presence of the pressure signal. All cells use fail-closed timeout semantics, a 10 ms deadline, and the same three low/medium/high pressure loads. Each cell contains 30 decisions in each of five independently started JVMs, for 1,800 raw observations. The high-pressure cell (16 attackers, 2,000 requests/s per attacker, 5 ms hold) is shown below; confidence intervals over the five JVM runs are retained in the artifact CSV.

At high pressure, shared capacity without a signal timed out in every decision and filled its queue. Enabling the signal reduced the timeout rate to 15.3% and the queue p95 to 13.1, showing that admission control can protect a shared executor but does not provide the same guarantee as reservation. Both reserved configurations had zero timeouts and an empty permission queue. Thus the ablation separates the two mechanisms: the signal reduces offered tenant work, whereas reserved capacity prevents that work from consuming permission capacity in the first place.

Reproducibility. The entry point is `PermissionStarvationJvmBenchmark`; the Docker-compatible `BenchmarkLauncher` runs the same JUnit method without Maven. Each run receives a unique `run_id`, writes a raw JSONL file and an environment record, and never overwrites an earlier run. The scripts `analyze_computing_experiments.py` and `analyze_qos_experiments.py` regenerate the CSV summaries and LaTeX macros used here; the cross-run CSV also reports 95% t intervals over independent JVM runs in addition to min/max ranges. `LlmTraceReplayRunner` and `analyze_llm_replay.py` perform the fixed-trace

Table 4: Core burst-lock workload at a 10 ms permission deadline.

	Windows		Ubuntu/KVM VPS	
Architecture	Timeout	Secure	Timeout	Secure
Layer-separated + fail-open	100.0%	0.0%	100.0%	0.0%
Layer-separated + fail-closed	100.0%	100.0%	100.0%	100.0%
Isolated, no coupling	0.0%	100.0%	0.0%	100.0%
Neuron coupled	0.0%	100.0%	0.0%	100.0%

application replay. The 2x2 ablation is launched with `-Dneuron.benchmark.mode=ablation_2x2` and summarized by `pythonevaluation/analyze\_ccpe\_ablation.py`; its input is restricted to the five run IDs `ccpe_ablation_01` through `ccpe_ablation_05`. The process-separated experiment is launched with `pythonevaluation/run\_permission\_http\_fault\_injection.py`; it starts the Java 21 service as a child process, writes immutable raw JSONL, and is summarized by `pythonevaluation/analyze\_permission\_http.py`.

5.3 RQ1: Starvation and Timeout Semantics

Table 4 reports the 10 ms burst-lock point averaged over the five independent core runs on each host. Percentages are calculated per run before averaging; they are not 30 independent samples from one JVM.

The two shared configurations have the same scheduling failure but different security meanings: fail-open converts a missed deadline into an unsafe admission, whereas fail-closed preserves safety by rejecting the request. Both reserved configurations avoid the measured timeout region on both hosts. This rejects the stronger claim that resource pressure alone defeats a permission layer; the failure requires a shared capacity bottleneck and a permissive timeout rule.

Answer to RQ1. The Java-native evidence reproduces a conditional policy-plane failure on two environments and across 3--50 ms deadlines. Availability is governed by capacity isolation; safety under starvation is governed by the fallback semantics.

5.4 RQ2: What Coupling Adds Beyond Isolation

For the depth-1 destructive probe, the static policy returns ASK-WITH-CONTEXT. When the resource rejection count exceeds the frozen threshold of 50, the coupled call to `EscalationPolicy.evaluate` tightens the boundary and returns DENY-DEPTH; the isolated-but-uncoupled call retains ASK. This is a policy effect beyond executor reservation. A deployment that is already statically fail-closed for the probe does not require this particular tightening rule for safety, although the correlated pressure signal remains useful for audit and incident response.

The factorial ablation separates the scheduling contribution from this policy contribution. At high pressure, shared capacity without a signal timed out in 100% of decisions, whereas enabling the signal reduced timeout to 15.3%; both reserved cells remained at zero timeout. The signal therefore reduces offered work and queue growth, while reservation protects permission capacity independently of the signal.

Answer to RQ2. Reservation explains timely decisions. Coupling changes one runtime-policy verdict under the same reserved capacity, and the 2x2 ablation shows that its admission effect is distinct from reservation. The two mechanisms are experimentally distinguishable and complementary.

5.5 RQ3: Independent Environment and Capacity Sensitivity

The second environment is an Ubuntu 22.04 Java 21 container on a one-vCPU KVM VPS with a 512 MB JVM heap limit. It is intentionally modest and is not presented as an independent physical server. The five core runs reproduce the same qualitative separation shown in Table 4. The local capacity sweep contains 9 configurations. At 10 ms, coupled governance remains timeout-free across the sweep; its secure decision rate is 97.1% in these stressed local capacity cells, while the isolated configuration is 93.3% secure because capacity reduction can cause benign policy probes to be classified conservatively. The fail-open and fail-closed controls expose the expected safety/availability trade-off.

These results show that the qualitative mechanism is not an artifact of one 3 ms setting or one host. They do not establish a universal optimal pool size: the VPS is a single virtualized environment and the capacity sweep is local.

Independent threshold holdout. The threshold was selected only from the training split using the preregistered candidate set and was then frozen at 50. As a low-cost post-selection check, we ran the Java 21 benchmark again on the Ubuntu/KVM VPS with six previously unused profiles: three attack profiles (alternating burst, staggered ramp, and long-hold drip) and three benign trajectories (tenant batch, cooperative burst, and background ramp). Each profile contributes 30 observations, for 180 observations in total. The fixed threshold detected all 90 attack observations (100.0%, 95% Wilson CI [95.9%, 100.0%]) and produced no false positives on 90 benign observations (0.0%, upper 95% Wilson bound 4.1%). This is an independent transfer check, not a claim that the threshold is universally optimal; the profiles are synthetic Java workload trajectories and remain subject to the VPS resource envelope.

Answer to RQ3. The failure/mitigation ordering is stable across the two recorded hosts and remains interpretable under nine capacity settings. Absolute latency and the secure-decision boundary remain deployment parameters.

5.6 RQ4: Process-Separated Permission Service

The local executor experiment alone cannot distinguish a JVM scheduling failure from a failure at a service boundary. We therefore start a second, independent Java 21 JVM running the Java `EscalationPolicy.evaluate` behind a bounded HTTP server. The client driver uses a direct loopback connection with proxy discovery disabled; the server and client are separate operating-system processes. Each cell contains 80% benign read requests and 20% depth-2 destructive requests, plus a background pressure stream. The service has two worker threads and an 8-entry queue in the pressure scenario and a 32-entry queue otherwise.

The process-separated matrix contains eight scenarios: nominal 10 and 50 ms deadlines, CPU/queue pressure, 5 ms service delay with a 10 ms deadline, 20 ms service delay with 10 and 50 ms deadlines, 50 ms delay at 50 ms, and a deterministic 1.5 s service pause. Five independent driver runs on the Ubuntu/KVM VPS produce 3,200 observations. Raw JSONL files, environment records, and the analysis manifest are retained under `evaluation/target/permission_http_remote/`; earlier Windows runs are retained separately and are not pooled with this VPS result.

At the 20 ms/50 ms point, fail-open secures only 35.0% of attack requests, whereas all three fail-closed variants preserve the attack decision. Neuron's coupled path reduces the timeout rate to 56.0% and retains an attack secure rate of 100.0%, but its benign completion is 30.0%. Coupling

Table 5: Process-separated permission service under a 20 ms service delay and 50 ms deadline. Values are means over five independent runs; each cell contains 16 benign and 4 attack requests.

Architecture	Attack secure	Benign complete	Timeout
Layer-separated + fail-open	35.0%	22.5%	75.0%
Layer-separated + fail-closed	100.0%	26.2%	72.0%
Isolated, no coupling	100.0%	21.2%	74.0%
Neuron coupled	100.0%	30.0%	56.0%

Table 6: Mixed-load QoS on Windows (100 requests per architecture/mix cell).

	90% benign / 10% attack		50% benign / 50% attack	
Architecture	Benign complete	Attack secure	Benign complete	Attack secure
Fail-open	0.0%	0.0%	0.0%	0.0%
Fail-closed	0.0%	100.0%	0.0%	100.0%
Isolated	100.0%	100.0%	100.0%	100.0%
Coupled	100.0%	100.0%	100.0%	100.0%

limits unsafe admission during pressure; it does not make a separated service faster than the deadline.

The pause scenario gives the clearest semantic separation. Fail-open reaches only 0.0% attack security with 100.0% timeouts, while the isolated, fail-closed, and coupled configurations retain 100.0% attack security and 100.0% benign completion in this controlled pause. These results support a process-boundary claim, but not a claim that coupling eliminates service partitions.

Answer to RQ4. The independent service experiment confirms that the safety result is not an artifact of the retired Python lock model or an in-process-only policy call. It also exposes the required trade-off: service delay and pause require capacity reservation and explicit fail-closed semantics; coupling is an additional admission guard, not a substitute for service availability.

5.7 RQ5: Benign QoS and API-Trace Replay

Mixed-load QoS. The dedicated QoS mode sends 100 permission requests per cell at a 10 ms deadline and controls the benign fraction at 100, 90, 75, or 50%. The 1,600 observations are from one Windows run and are reported separately from the cross-host core count. At the 90/10 mix, isolated and coupled governance complete 100% of benign requests and securely handle 100% of attack requests; both shared controls suffer policy timeouts. At the 50/50 mix, the same separation remains, while fail-open exposes unsafe attack admissions and fail-closed preserves attack safety by sacrificing benign completion. This directly measures QoS rather than treating rejection count as a security benefit.

API-derived trace replay. The original API collection contains 33 parseable sessions: 18 benign sessions (78 tool calls) and 15 resource-abuse sessions (176 tool calls). The Java replay evaluates the same 254 immutable calls under all four architectures, producing 1016 architecture evaluations. The reserved isolated and coupled paths complete 100% of both workload classes; the shared paths time out under the replay harness's 10 ms local policy deadline. Because the model responses are not regenerated, this is an integration check using real tool-call shapes, not a claim of statistical coverage of the LLM ecosystem. We additionally replayed the same 33 sessions on the Ubuntu/KVM VPS,

producing the same 1,016 architecture-level call observations without regenerating model responses; this checks that the replay path is not Windows-specific.

Answer to RQ5. The coupled architecture preserves benign completion while retaining the attack decision in the measured mixed-load and API-trace settings. Residual risk lies in deadline and capacity calibration, audit backpressure, and failures outside the local JVM/container trust boundary.

5.8 Threats to Validity

Internal validity. JIT compilation, garbage collection, VPS co-tenancy, and scheduler noise can influence latency. We warm each cell, retain untrimmed observations, report environment records, and use independent JVM runs. The rate limiter and escalation policy are runtime classes; workload drivers and timeout wrappers are experimental harness code.

Construct validity. A missed 10 ms harness deadline represents policy-plane unavailability, not proof that a deployed service uses that deadline. We therefore compare both fallback semantics and report the deadline sweep. Resource rejection count is a pressure signal, not a direct measurement of CPU, memory, or lock contention. The QoS mixture controls request classes, while background attack load is a synthetic stressor.

External validity. The second host is a one-vCPU virtual machine, not a second physical microarchitecture. The process-separated service is still a loopback deployment on one host; it does not establish WAN latency, multi-node, GPU, or live-traffic performance. Capacity sensitivity is local, and the API replay uses one previously collected trace set. Audit-queue partitions remain outside the measured matrix. AutoGen, LangGraph, and MCP are not security-system baselines.

6 Related Work

Agent runtimes and orchestration. AutoGen, LangGraph, and related systems provide abstractions for agent coordination, tool invocation, and workflow state [2, 3]. MCP standardizes communication between model-facing clients and tool/data servers [4]. These projects are relevant integration contexts, not equivalent security baselines: their primary goals differ from multi-tenant resource and permission governance. Neuron can sit below an orchestrator or beside a protocol gateway. Our comparison is therefore architectural---which identity, pressure, and verdict signals must cross the boundary---rather than a performance ranking against those projects.

AIOS-like work moves scheduling, memory, and tool management into an agent-oriented systems layer [1]. Neuron shares the premise that agents require runtime support, but focuses on concurrent executor behavior and permission availability. The key distinction is not whether a framework has a rate limiter or an access-control list in isolation; it is whether policy-plane capacity, queueing, and timeout semantics remain safe under tenant pressure and whether pressure can be part of a permission decision when desired.

Concurrent runtime and overload control. Operating-system schedulers, bounded executors, containers, and cgroup-based resource controls provide the primitives for isolating CPU and memory demand [9, 5, 6]. Overload control and circuit breaking similarly distinguish admission, queueing, and fallback behavior. These mechanisms explain why a reserved permission executor is a reasonable

systems baseline. Neuron studies the remaining coupling question: whether the same runtime pressure should also change a permission verdict for a request that has not yet crossed its static depth boundary.

Recent agent-security surveys and benchmarks study policy compliance, tool misuse, and interaction-level safety [10, 11, 12, 13]. These works motivate runtime enforcement, but do not evaluate whether tenant-induced executor contention makes the authorization path unavailable or how timeout semantics convert that availability loss into an admission decision.

Resource isolation and admission control. Operating systems and containers provide CPU, memory, process, and I/O isolation. Token buckets and cgroups are established mechanisms for limiting bursts and allocating capacity. A deployment can place its permission service in a separate process or cgroup, which is a strong alternative to Neuron's reserved executor. Our isolated baseline reflects the same design principle inside one JVM. The paper does not claim that cross-layer coupling replaces isolation: the evaluation shows that isolation protects availability, whereas coupling supplies a pressure-conditioned verdict.

Work on overload control and circuit breaking similarly distinguishes admission, queuing, and fallback behavior. Neuron's failure model applies that distinction to a security-sensitive policy path. A fail-open fallback trades security for availability; a fail-closed fallback makes the opposite trade. Making both baselines explicit is essential because lower timeout correctness in a fail-open system is otherwise easily misattributed to a new security mechanism.

Access control, delegation, and confused deputies. Privilege attenuation and the confused-deputy problem have long motivated capability systems and least-privilege delegation [14]. Neuron's spawn-time privilege non-increase applies that principle to dynamic sub-agent creation: a child cannot silently receive a more permissive profile than its parent. Depth is an additional, agent-specific risk signal, not a substitute for capabilities or target-scoped authorization. The pressure-aware rule further treats runtime state as context for a narrow class of destructive requests.

Policy availability and fail-safe defaults. Security engineering commonly recommends fail-safe defaults, but real systems sometimes introduce permissive fallbacks to meet latency or availability targets. Prior work on reference monitors emphasizes complete mediation and tamper resistance; distributed authorization work also recognizes policy-service latency and partitions. Our contribution is a controlled characterization of this tradeoff in a multi-tenant agent runtime, including an isolated fail-closed baseline. We do not claim the general discovery that overload can delay a service. The contribution lies in tracing that delay through agent permission semantics and evaluating a resource-informed policy response.

Policy-as-code engines such as Open Policy Agent (OPA) [15] provide an important practical comparator: they separate declarative policy evaluation from application code and can be deployed beside an API gateway or admission controller. OPA, identity-oriented authorization, and relationship-based authorization are complementary policy mechanisms; none by itself specifies reserved executor capacity or whether a deadline miss under tenant pressure fails open. Neuron evaluates that runtime availability and verdict boundary explicitly rather than claiming to replace these policy engines.

Cross-layer observability. Distributed tracing correlates events across services through propagated identifiers. Neuron uses the same systems idea across resource admission, spawn, and permission paths. Correlation alone does not enforce safety; it makes the causal chain inspectable

and supplies consistent context to the joint decision. The enforcement contribution is the policy input and its reserved execution path, not the trace identifier by itself.

Positioning for concurrent systems research. The work is a software-architecture study of a concurrent runtime control plane. Its unit of comparison is an architecture configuration---shared fail-open, shared fail-closed, isolated, or coupled---and its main outcomes are executor availability, queueing, and permission-verdict semantics. Sandboxing, WASM, Docker, LLM routing, and formal models are useful components of the broader Neuron prototype, but they are outside the causal evaluation and are not presented as independent innovations here.

7 Conclusion

Resource and permission decisions are not independent when they share a concurrent runtime. This paper isolates a precise failure condition: hostile work can fill a shared executor and delay a policy decision, while a permissive deadline fallback can turn that queueing delay into a fail-open window. The condition is neither inevitable nor universal. Fail-closed handling preserves safety, and reserved policy-plane capacity addresses the concurrency bottleneck.

Neuron combines that reservation with a second mechanism: a resource signal can tighten the permission policy during sustained pressure. The four-way comparison makes the attribution explicit. Isolation explains policy-plane availability; coupling explains the changed depth-1 verdict. Identity propagation, privilege non-increase, and correlated auditing support this decision path, while quota and sandbox components are engineering context.

The revised evidence runs in the Java 21 Neuron runtime and replaces the former Python lock model. It uses independent JVM replications on a Windows host and an Ubuntu/KVM container, sweeps 3--50 ms deadlines and attacker pressure, evaluates nine capacity configurations, and measures benign/attack QoS mixtures. A fixed replay of 33 API-derived sessions checks that the same decision path handles application-shaped tool calls, and the same replay was also executed on the Ubuntu/KVM VPS. All reported counts are generated from retained JSONL by the analysis scripts. A further five-run, 3,200-observation experiment on the Ubuntu/KVM VPS starts the permission policy in a separate Java 21 process and injects service delay, queue pressure, and a deterministic pause. This separates a process-boundary fault from the retired in-process Python lock model.

An additional independent Windows core run records executor-oriented observables at the same pressure factors. At the highest offered concurrency, the shared configurations fill the permission queue and miss the deadline, whereas reserved capacity keeps the measured permission queue empty. This supports the concurrency interpretation without attributing the queueing improvement to cross-layer coupling.

The evidence supports a conditional architecture claim within a JVM/container trust boundary. It does not establish that every agent framework fails open, that the selected threshold transfers unchanged to production, or that a one-vCPU VPS represents all hardware. The process- separated result is still a loopback, one-host experiment; WAN partition behavior, audit-queue exhaustion, and genuinely live multi-tenant traffic remain future work. Until then, Neuron is a reproducible concurrent-runtime mechanism and an argument for making resource--policy interactions explicit in multi-tenant agent services.

8 Ethics and Reproducibility

Ethics. All experiments were conducted against the authors' own runtime; no third-party users or personally identifiable data were involved. The API collection used commercial LLM endpoints under their respective terms of service, with no personally identifiable information processed and adversarial prompts confined to the controlled experimental context.

Reproducibility. The evaluation harness, probe payloads, capability-surface classifier, TLA$^+$ specification, and raw result files (CSV/JSON) are released as a replication package to enable independent verification. The package contains the Java source, environment records, raw JSONL observations, analysis scripts, generated manuscript values, and a one-click consistency check. The process-separated permission-service experiment is explicitly a same-host loopback deployment and is not presented as WAN or multi-node evidence.

Statements and Declarations

Funding. No funding was received for conducting this study or for preparing this manuscript.

Competing interests. The authors declare that they have no relevant financial or non-financial interests to disclose.

Ethics approval and consent to participate. Not applicable. The study used the authors' own software runtime and did not involve human participants, animals, or personally identifiable information.

Consent for publication. Not applicable.

Data availability. The raw JSONL observations, environment records, derived summaries, and frozen manuscript values are included in the public replication package at `https://github.com/dadawer-web/ouisani/tree/ccpe-repro-20260811/ccpe-repro-20260811`. The package is also prepared for a future Zenodo deposit; a DOI can be added after deposit without changing the reported data.

Code availability. The Java 21 Neuron source, benchmark drivers, analysis scripts, formal model, and one-click reproduction instructions are included in the same public repository and are released under an open-source license.

Author contributions. Mingyuan Xie and Zhengxun Wu jointly contributed to the conception, methodology, software implementation, experiments, analysis, and manuscript preparation. Both authors approved the submitted version and agree to be accountable for the work.

A Artifact and Reproduction Protocol

The artifact is organized so that paper values are derived from raw runtime observations rather than copied into tables by hand.

1. From `neuron-java`, run `mvn -Dtest=PermissionStarvationJvmBenchmark test` with Java 21; pass a unique `-Dneuron.benchmark.runId` and `-Dneuron.benchmark.output` for every JVM invocation.

2. Confirm that `evaluation/target/jvm_starvation_new/` contains the raw JSONL, summary CSV, and environment records for both hosts.

3. Run `pythonevaluation/analyze_computing_experiments.py` and `pythonevaluation/analyze_qos_experiments.py`. These aggregate per run, generate the cross-run CSVs, and write the TeX macros used by the paper. Run `pythonevaluation/analyze_llm_replay.py` after `LlmTraceReplayRunner` to regenerate the trace-replay macros.

4. Run `pythonevaluation/analyze\_ccpe\_concurrency.py` to regenerate the executor-oriented pressure surface and the macros used in Table 2. The additional Windows run is identified by `ccpe_core_01` and remains separate from the original cross-environment count.

5. Run five independent JVMs with `-Dneuron.benchmark.mode=ablation_2x2`, using run IDs `ccpe_ablation_01` through `ccpe_ablation_05`; then run `pythonevaluation/analyze\_ccpe\_ablation.py` to regenerate the factorial summary and Table 3 macros.

6. Run `pythonevaluation/run\_permission\_http\_fault\_injection.py` with `-mode core -decisions 20 -run-id <id>` to launch the process-separated permission service. After the five independent runs, regenerate its values with `pythonevaluation/analyze\_permission\_http.py` and the pattern `permission_http_vps_http_prod_*.raw.jsonl`.

7. Run `pythonevaluation/analyze\_threshold\_holdout.py` after the frozen Java holdout JSONL is present. This script does not select a new threshold; it only evaluates the fixed value 50 on the six unseen profiles.

8. Build the CCPE paper from `addtions/paper_ccpe`. The generated TeX file is imported by `main.tex`; a missing file therefore causes the consistency check to fail rather than silently substituting an old value.

The benchmark accepts `-Dneuron.benchmark.decisions=N` for diagnostic runs. Paper core results use $N = 30$ per cell; the QoS mode uses $N = 100$ to approximate the requested mixture proportions. Hardware replications should preserve the factor grid and add an environment-specific output directory rather than overwrite the first run.

B Factor Grid and Recorded Fields

The core deadline sweep uses $\{3, 10, 50\}$ ms. The pressure surface uses attacker count $\{1, 4, 16\}$, per-producer request rate $\{100, 500, 2000\}$ requests/s, and hold time $\{0.5, 2, 5\}$ ms. Each raw record contains the architecture, split, workload class, morphology, trial, all factor values, latency, timeout flag, queue-admission flag, verdict, secure-decision flag, cumulative resource rejections, executed attack-task count, and permission-queue depth. The separate `qos` mode records the 100/90/75/50 benign fractions, benign completion, and error tightening.

C Claims Deliberately Excluded from the Main Evaluation

The repository contains additional tests for sandboxes, quotas, external orchestration libraries, LLM-generated attacks, and formal models. They are not inputs to the results in Section 5. In particular, the retired Python multiprocessing starvation output is retained only for historical comparison and must not be cited as evidence for the JVM claim. The current paper does not make a multi-node, GPU, or production-traffic performance claim. The process-separated permission fault experiment is evaluated on loopback; no WAN latency, partition, or remote multi-node service claim is made. The API sessions are replayed traces, not newly sampled online sessions.

References

[1] K. Mei, X. Zhu, W. Xu, W. Hua, M. Jin, Z. Li, S. Xu, R. Ye, Y. Ge, Y. Zhang, in *Proceedings of the Fourth International Conference on Language Modeling (COLM)* (2025). ArXiv:2403.16971

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A.H. Awadallah, R.W. White, D. Burger, C. Wang, arXiv preprint arXiv:2308.08155 (2023)

[3] LangChain. LangGraph: Building stateful, multi-actor applications with LLMs. `https://github.com/langchain-ai/langgraph` (2024)

[4] Anthropic. Model context protocol. `https://modelcontextprotocol.io` (2024)

[5] Linux Kernel Documentation. cgroup v2 --- linux kernel documentation. `https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html` (2024)

[6] A. Young, M.D.R. Kiper, V. Gopalakrishnan, R. Pal, D. Shetty, P. Aronoff, A. Belay. gVisor: A container sandbox runtime providing a security isolation layer for containers. `https://gvisor.dev` (2023)

[7] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, D.M. Popa, in *17th Workshop on Hot Topics in Operating Systems (HotOS)* (2019). Extended version: USENIX ATC 2020

[8] R.N.M. Watson, J. Anderson, B. Laurie, K. Kennaway, in *Proceedings of the 19th USENIX Conference on System Administration (LISA)* (2010). Capability-based sandboxing in FreeBSD

[9] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, S. Winwood, in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)* (2009)

[10] Y. Wang, D. Xue, N. Zhang, W. Zhang, in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP) Findings* (2024)

[11] Y. Ruan, X. Zhang, X. Chen, C. Zhang, Y. Wu, X. Yue, in *Proceedings of the 12th International Conference on Learning Representations (ICLR)* (2024). ToolEmu: emulates tool execution to identify risks of LLM agents without real-world side effects; sandboxing at the emulation layer

[12] Z. Zhang, J. Lu, X. Lyu, S. Zhang, X. Peng, H. Fei, H. Lin, X. Zou, K. Wang, Y. Zhang, Q. Ge, T. Xiao, T. Hu, Y. Zheng, J. Liu, Z. Wang, Z. Li, C. Liu, J. Li, H. Zhu, Z. Liu, Y. Zhang, K. Sun, Y. Zhang, T. Gui, Q. Zhang, X. Huang, in *arXiv preprint arXiv:2412.14470*

(2024). Benchmark of 10K test cases across 8 safety risk types for LLM agents; evaluates both interaction-level and tool-level safety

[13] L. Struppek, D. Hintersdorf, F. Friedrich, K. Kersting, in *Proceedings of the 33rd USENIX Security Symposium* (2025). ArXiv:2407.11809

[14] N. Hardy, ACM SIGOPS Operating Systems Review **22**(4), 36 (1988). Origin of the confused-deputy problem; capability-based authority propagation

[15] Open Policy Agent Authors. Open Policy Agent: An open source, general-purpose policy engine. `https://www.openpolicyagent.org` (2024). CNCF graduated policy-as-code engine; uses the Rego declarative policy language for stateless authorization evaluation across API gateways, admission controllers, and CI/CD pipelines